CÓDIGOCLARO · MATERIAL DE ESTUDIO

Ruta TypeScript, JavaScript, React y Node: 15 temas

Tipos estrictos, migración gradual, módulos de servidor y componentes React

1. TypeScript: tipado, inferencia y configuración estricta

Objetivo: Entender que TypeScript comprueba tipos en desarrollo y que sus anotaciones no validan datos durante la ejecución.

TypeScript añade análisis estático al código JavaScript y emite JavaScript; los tipos se eliminan al compilar y no son un sistema de validación en runtime. El compilador puede inferir tipos desde inicializadores y contexto, por lo que no hace falta anotar cada variable. Activa `strict` para detectar más errores antes de ejecutar y revisa las opciones del proyecto en `tsconfig.json`. `noUncheckedIndexedAccess` ayuda a modelar que acceder a un índice puede no encontrar valor. Fija la versión del compilador y las definiciones de tipos con el lockfile; un `as Tipo` silencia el análisis, no convierte ni comprueba un objeto.

Código · ejemplo

```
[TypeScript 7.x]
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "target": "ES2022"
  }
}
```

El compilador usa tipos para detectar usos incompatibles y después emite JavaScript; una anotación no modifica el valor recibido. strict activa comprobaciones como strictNullChecks, y noUncheckedIndexedAccess modela que un índice puede no existir. La versión instalada y tsconfig efectiva forman parte del ejercicio: ejecuta el comando local y revisa qué configuración leyó. Las aserciones `as` desaparecen y nunca deben confundirse con parseo o validación de datos.

Práctica: Una API devuelve texto JSON. ¿Por qué no basta con escribir `const user = data as User`?

> Respuesta razonada: La aserción solo cambia la opinión del compilador; no examina el texto recibido y el objeto podría carecer de propiedades o tener tipos incorrectos. Analiza JSON como `unknown` y valida sus campos en runtime antes de tratarlo como `User`. Comprueba que el proyecto usa el compilador esperado y que la configuración estricta realmente se carga.

Documentación consultada: TypeScript · Handbook · TypeScript · TSConfig strict · TypeScript · Release 7.0

2. Tuplas readonly y propiedades opcionales

Objetivo: Representar posiciones conocidas y propiedades que pueden faltar sin hacer mutaciones accidentales.

Una tupla asigna un tipo a cada posición y es útil para un resultado pequeño de forma fija, aunque una interfaz con campos puede ser más legible si el valor crece. `readonly` evita mutaciones desde esa referencia durante la comprobación estática, pero no congela el objeto en runtime. Una propiedad opcional `name?: string` puede no existir; con `exactOptionalPropertyTypes`, asignar `undefined` explícito se distingue mejor de omitir el campo. Antes de leer un valor opcional, comprueba su presencia o aplica una regla de presentación. No uses `!` para esconder la posibilidad de ausencia sin una garantía real.

Código · ejemplo

```
[TypeScript]
type Result = readonly [ok: boolean, message?: string];
interface Profile {
  displayName?: string;
}
```

Una tupla expresa posiciones y tipos fijos; para una respuesta que crezca, un objeto con campos suele ser más fácil de leer que índices numéricos. readonly impide mutaciones permitidas por el tipo de esa referencia, pero otra referencia todavía puede cambiar el objeto. Con exactOptionalPropertyTypes, propiedad omitida y valor undefined explícito son casos distintos. Define el contrato de serialización y prueba objeto ausente, campo vacío y campo presente con valor normal.

Práctica: ¿Cómo distinguir “no se envió `displayName`” de un nombre vacío?

Respuesta razonada: Comprueba si la propiedad existe y define por separado si `""` es válida. La opción `exactOptionalPropertyTypes` evita tratar automáticamente el campo omitido como idéntico a un campo asignado `undefined`, pero no convierte una cadena vacía en ausencia. La regla pertenece al contrato de la API y debe reflejarse también en validación de runtime.

Documentación consultada: TypeScript · Handbook

3. Uniones discriminadas y narrowing

Objetivo: Modelar varios estados posibles y reducir una unión mediante comprobaciones seguras.

Una unión describe que un valor puede tomar uno de varios tipos. Añadir una propiedad literal como `kind` permite que TypeScript estreche el tipo en cada rama del `switch`. El narrowing se basa en comprobaciones como `typeof`, igualdad de literales, `in` y guardas de tipo. Una comprobación incompleta puede dejar un estado sin tratar; una función exhaustiva con `never` ayuda a que la incorporación de una nueva variante provoque un error útil. Una unión estática solo es cierta si la entrada fue validada al cruzar la frontera externa. Evita tipos con muchos campos opcionales que permitan combinaciones imposibles.

Código · ejemplo

```
[TypeScript]
type Load<T> =
  | { kind: "loading" }
  | { kind: "ready"; value: T }
  | { kind: "error"; message: string };
```

Una unión discriminada agrega un campo literal como kind para que el compilador estreche el tipo dentro de cada rama. Después del switch, una variable never puede comprobar que se trataron todos los estados. Si se añade un nuevo tipo, el error de compilación guía los consumidores que faltan. No mezcles datos listos con un mensaje de error opcional si eso permite estados contradictorios; expresa las variantes válidas en el tipo.

Práctica: ¿Qué debería hacer la interfaz en cada variante y cómo detectar una futura variante nueva?

Respuesta razonada: Muestra indicador de carga, representa `value` cuando está lista y presenta un mensaje recuperable cuando falla. En un `switch` que termina con una comprobación `const exhaustive: never = state`, añadir una variante obliga a revisar el renderizado. Los valores procedentes de JSON aún necesitan validación antes de entrar a este tipo.

Documentación consultada: TypeScript · Handbook

4. Funciones, callbacks y type frente a interface

Objetivo: Dar firmas explícitas a funciones y elegir una forma de declarar contratos que facilite su uso.

Una firma de función define parámetros y retorno; si el contexto permite inferirlos, no repitas anotaciones sin aportar claridad. Las funciones callback deben expresar qué argumentos reciben y cuándo se ejecutan; comprueba si una API los llama una o varias veces y si permite una operación asíncrona. `type` puede describir uniones, primitivas, tuplas y alias de firma; `interface` describe con claridad la forma de un objeto y admite extensión/declaración combinada. No existe una regla universal de que una forma sea siempre mejor. Elige por la forma del contrato y mantén consistencia en el proyecto.

Código · ejemplo

```
[TypeScript]
type Validator<T> = (value: unknown) => value is T;
interface User {
  id: string;
  name: string;
}
const isUser: Validator<User> = (value): value is User =>
  typeof value === "object" && value !== null && "id" in value && "name" in value;
```

Una función se tipa por parámetros y retorno; un callback expresa qué argumentos recibe. `interface` describe y extiende la forma de objetos; `type` también representa uniones y tuplas. Usa inferencia si conserva el contrato y anota cuando aclara una ambigüedad.

Práctica: ¿Qué comprobaciones faltan para que `isUser` verifique completamente sus propiedades?

Respuesta razonada: La guarda solo verifica que existan `id` y `name`, pero no que ambos sean cadenas. Debe comprobar `typeof record.id === "string"` y lo mismo para `name`, tras confirmar que el valor es un objeto no nulo. En producción, contempla arrays y propiedad heredada según el formato esperado; la guarda debe garantizar realmente todo el contrato que anuncia.

Documentación consultada: TypeScript · Handbook

5. Null, unknown y límites de confianza

Objetivo: Tratar valores ausentes y datos externos sin recurrir a `any` ni a aserciones indiscriminadas.

Con `strictNullChecks`, `null` y `undefined` son valores diferentes de `string`, `number` u otros tipos. Antes de usar un posible valor ausente, comprueba la condición o proporciona un comportamiento alternativo deliberado. `unknown` permite recibir cualquier dato, pero obliga a estrechar su tipo antes de operar; `any` desactiva buena parte de esa protección y propaga errores. Las comprobaciones de tipo en compilación no leen datos de red, almacenamiento local ni formularios. Define funciones pequeñas de validación en los límites, y modela opcionales según el formato real, no para evitar investigar por qué falta un dato.

Código · ejemplo

```
[TypeScript]
function displayName(name: string | null | undefined): string {
  return name?.trim() || "Sin nombre";
}
function parseInput(value: unknown): string | undefined {
  return typeof value === "string" ? value : undefined;
}
```

unknown obliga a comprobar el dato antes de usarlo, mientras any permite cualquier operación y propaga la pérdida de seguridad. Tras typeof value === "object", todavía hay que excluir null y confirmar las propiedades. La estrechez de tipo solo es confiable para el flujo que el compilador puede analizar; datos de JSON requieren validación de cada campo en runtime. Mantén separados el error de transporte y el rechazo del esquema.

Práctica: ¿Qué diferencia hay entre `??` y `||` al dar un valor por defecto?

Respuesta razonada: `??` aplica el valor alternativo solo ante `null` o `undefined`; `||` también lo hace con `""`, `0` y `false`. Si cero o cadena vacía pueden ser datos válidos, la elección cambia el resultado. El comportamiento debe corresponder al contrato y se cubre con pruebas para esos valores frontera.

Documentación consultada: TypeScript · Handbook · TypeScript · TSConfig strict

6. Genéricos y tipos derivados

Objetivo: Reutilizar funciones sin perder la relación entre sus entradas y salidas y derivar tipos de estructuras existentes.

Un parámetro genérico conserva información del tipo concreto entre entrada y salida. Para limitar operaciones disponibles, añade una restricción como `T extends { id: string }`; no afirmes más de lo que garantiza. Los tipos derivados con `Pick`, `Omit`, `Partial` y `Readonly` evitan copiar formas, pero pueden crear modelos de actualización demasiado permisivos si se aplican sin revisar reglas. Los genéricos solo existen durante el análisis de TypeScript: no seleccionan un parser en runtime. Infiera tipos desde argumentos cuando sea posible. Si una transformación cambia datos o descarta valores, refleja esa regla en el retorno.

Código · ejemplo

```
[TypeScript]
function byId<T extends { id: string }>(items: readonly T[]): Map<string, T> {
  return new Map(items.map(item => [item.id, item] as const));
}
type UserPatch = Partial<Pick<User, "name">>;
```

Un parámetro genérico T conserva la relación entre entrada y salida; evita repetir una versión para cada tipo sin perder información. Una restricción `T extends { id: string }` permite leer id porque todos los tipos admitidos lo declaran. Si el tipo de salida no depende de la entrada, el genérico quizá no aporta valor. Prueba inferencia con dos tipos distintos y evita recurrir a any para silenciar una restricción mal diseñada.

Práctica: ¿Qué garantía aporta `T extends { id: string }` y qué no garantiza sobre datos recibidos por HTTP?

> Respuesta razonada: Permite leer una propiedad `id` de tipo string en cada `T`, pero no valida un objeto JSON externo. Tampoco garantiza que los IDs sean únicos; si se repiten, el mapa conserva el último valor. Valida entradas y decide explícitamente cómo tratar duplicados.

Documentación consultada: TypeScript · Handbook

7. Clases y estado válido

Objetivo: Usar clases cuando identidad o estado mutable encapsulado aportan valor, manteniendo invariantes verificables.

Las clases crean instancias con estado y métodos. Un constructor debe dejar el objeto válido o fallar con un error comprensible; no repartas validación entre setters que permitan estados intermedios ilegales. Los campos `readonly` limitan asignaciones posteriores desde código TypeScript, pero no equivalen a inmutabilidad profunda en runtime. Los campos `#private` de JavaScript proporcionan privacidad de lenguaje; la palabra `private` de TypeScript es principalmente una restricción estática. Prefiere funciones y tipos de objeto sencillos para datos sin comportamiento propio. Añade herencia solo cuando la relación sustituible sea clara; composición suele facilitar cambios.

Código · ejemplo

```
[TypeScript / ECMAScript]
class Enrollment {
  readonly createdAt = new Date();
  #status: "active" | "cancelled" = "active";
  cancel() { this.#status = "cancelled"; }
}
```

El constructor crea un estado inicial y los métodos públicos deben conservar las invariantes. readonly impide reasignar una propiedad durante comprobación estática; `private` de TypeScript también es una restricción del compilador, mientras los campos `#privados` tienen semántica de runtime de JavaScript. Valida valores externos antes de construir la instancia. Decide si una clase aporta identidad y comportamiento o si un objeto de datos y funciones resulta más sencillo.

Práctica: ¿Qué evita `#status` y por qué la propiedad `readonly createdAt` no congela el objeto Date?

> Respuesta razonada: `#status` es un campo privado de JavaScript y no puede leerse directamente desde fuera de la clase. `readonly` impide reasignar la referencia durante el análisis TypeScript, pero `Date` sigue siendo mutable: métodos como `setTime` pueden cambiar su valor. Devuelve copias o usa una representación inmutable si el contrato lo exige.

Documentación consultada: TypeScript · Handbook

8. Node.js, módulos y versiones

Objetivo: Configurar módulos coherentes en Node y alinear las definiciones de tipos con el runtime instalado.

Node.js ofrece APIs de servidor distintas de las APIs del navegador, aunque ambos ejecuten JavaScript. Decide el sistema de módulos del proyecto y mantén coherentes `package.json`, `tsconfig` y extensiones emitidas; `NodeNext` sigue las reglas de módulos de Node según el campo `type`. Instala TypeScript y `@types/node` como dependencias de desarrollo con versiones fijadas en el lockfile y compatibles con el major del runtime desplegado. Los mecanismos nativos de Node para ejecutar TypeScript, si se usan, dependen de la versión y no hacen validación de tipos equivalente a compilar. Verifica scripts con la versión LTS objetivo y no mezcles APIs del navegador en código servidor.

Código · ejemplo

```
[Node.js 26 LTS + TypeScript]
{
  "type": "module",
  "devDependencies": {
    "typescript": "7.x",
    "@types/node": "26.x"
  }
}
```

Node mantiene versiones Current y LTS; la fecha de soporte cambia y debe comprobarse antes de fijar una versión de producción. package.json y tsconfig determinan cómo se resuelven módulos y qué sintaxis se emite. Un módulo que usa document pertenece al navegador y no debe ejecutarse directamente en Node. Fija runtime y gestor de paquetes, conserva lockfile y prueba el comando real de compilación y arranque.

Práctica: ¿Por qué un proyecto puede compilar localmente y fallar al ejecutar en producción?

> Respuesta razonada: Puede haber diferencias en major de Node, formato ESM/CommonJS, dependencias instaladas o archivos emitidos. El compilador comprueba tipos, pero no garantiza que el runtime tenga una API disponible. Fija versiones, reproduce el script de producción en CI y verifica la salida compilada bajo la misma versión principal de Node.

Documentación consultada: Node.js · API Documentation · Node.js · Release schedule

9. Migración gradual desde JavaScript

Objetivo: Añadir comprobaciones de TypeScript por etapas a un proyecto JavaScript sin reescribirlo todo.

La migración incremental puede empezar activando `allowJs` y `checkJs`, y usando comentarios JSDoc donde TypeScript ya no puede inferir suficiente. Renombrar un archivo a `.ts` no revela automáticamente sus supuestos ni valida datos externos. Migra primero módulos estables con interfaces claras y corrige errores por grupos pequeños. Mantén límites entre código migrado y legado, evita introducir `any` nuevo y registra por qué se conserva uno temporalmente. La configuración debe coincidir con el comando de compilación usado por el equipo. Añade pruebas alrededor de comportamiento actual para distinguir errores reales de cambios accidentales durante la tipificación.

Código · ejemplo

```
[tsconfig + JavaScript]
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": true,
    "strict": true
  },
  "include": ["src"]
}
```

allowJs y checkJs permiten que archivos existentes participen en comprobación; JSDoc puede documentar parámetros antes de renombrar cada archivo. Migra una frontera estable y añade pruebas de comportamiento para distinguir errores de tipos de cambios funcionales. Mantén cualquier any temporal localizado y con motivo; no lo propagues a las capas nuevas. Revisa la configuración que ejecuta CI, porque el editor puede estar comprobando un tsconfig distinto.

Práctica: ¿Qué valor aporta `// @ts-check` en un archivo JS y qué trabajo no reemplaza?

> Respuesta razonada: Pide al editor o compilador comprobar ese archivo JavaScript y ayuda a migrar con JSDoc. No transforma las entradas en tipos runtime, no actualiza las dependencias ni reemplaza pruebas funcionales. Actívalo en módulos seleccionados, anota contratos reales y resuelve los errores en lugar de silenciarlos globalmente.

Documentación consultada: TypeScript · Handbook · TypeScript · TSConfig strict

10. React TSX: props, eventos, estado y peticiones

Objetivo: Tipar componentes y eventos React, representar estados de carga y validar en runtime la respuesta de una API.

Un componente React recibe props como valores de entrada; `children` se tipa según si admite contenido. Los eventos usan tipos de React como `ChangeEvent<HTMLInputElement>` o `FormEvent<HTMLFormElement>`. El estado se infiere desde su valor inicial, pero una unión explícita ayuda si puede contener estados distintos. Actualiza arrays y objetos creando nuevas referencias; no los mutés en estado. En peticiones, representa carga, éxito y error como estados discriminados, cancela solicitudes obsoletas cuando corresponda y analiza respuestas desde `unknown`. React no valida el JSON remoto por sí mismo. Consulta la documentación de la versión React y alinea `@types/react` si el proyecto lo necesita.

Código · React + TypeScript

```
type User = { id: string; name: string };
type Load<T> = { kind: "loading" } | { kind: "ready"; data: T } | { kind: "error";
message: string };
function isUser(value: unknown): value is User {
  return typeof value === "object" && value !== null &&
    "id" in value && typeof value.id === "string" &&
    "name" in value && typeof value.name === "string";
}
function SubmitButton({ disabled }: { disabled: boolean }) {
  return <button type="submit" disabled={disabled}>Guardar</button>;
}
```

Práctica: Un servidor responde con `{ "id": 7, "name": "Ana" }`. ¿Debe mostrarse el estado ready si el id debe ser string?

Respuesta razonada: No. isUser rechaza el id numérico y el componente mantiene un estado de error con un mensaje comprensible. El tipo de TypeScript se borra al emitir JavaScript; solo el guard ejecuta una comprobación de runtime. Incluye además el estado loading y evita que una respuesta anterior reemplace la de la búsqueda actual.

Documentación consultada: React · TypeScript · React · Forms · TypeScript · Handbook

Fuentes oficiales

TypeScript · Handbook · TypeScript · TSConfig strict · TypeScript · Release 7.0 · Node.js · API Documentation · Node.js · Release schedule · React · TypeScript · React · Forms · ECMA · ECMAScript 2025